



Enhancing intrusion detection in IoT: CNN integration with K-means for efficient and balanced classification

Sarra Cherfi ^{a,b,*}, Ammar Boulaiche ^{a,b}, Ali Lemouari ^{a,c}, Abdelhafid Abouaissa ^d

^a LaRIA Laboratory, University of Jijel, Jijel, 18000, Algeria

^b Faculty of Exact Sciences and Computer Science, University of Jijel, Jijel, 18000, Algeria

^c Faculty of Science and Technology, University of Tamanrasset, Tamanrasset, 11000, Algeria

^d IRIMAS UR 7499, University of Haute Alsace, Mulhouse, France

ARTICLE INFO

Keywords:

Internet of things
Intrusion detection systems
Convolutional neural network
K-means algorithm

ABSTRACT

The fact that the Internet of Things (IoT) is rapidly spreading and is everywhere, helps to increase speed and enlarge the amount of data moving around on earth on a planetary scale. The growing number of connected units leads to the excess of the flow of traffic through the networks, and an ecosystem of dense and inter-networked digital environments rises. Nevertheless, this astronomical growth of network traffic is accompanied by a parallel surge in the possibility to have cyber attacks. Therefore, intrusion detection systems (IDSs) are increasingly needed to confront these attacks. In this paper, we propose a detection model based on a deep learning method called convolutional neural network (CNN). To train this model, two datasets are used, Canadian Institute for Cybersecurity Intrusion Detection System 2017 (CICIDS2017) and Telemetry and Network data for the Internet of Things (ToN-IoT). These datasets first go through a size reduction step, which aims to reduce the size of the training set to speed up the learning process, avoid the overlearning problem, and also solve the class-imbalanced problem. The method used in this stage is the k-means algorithm. The results obtained in this work demonstrate the effectiveness of our approach, which can achieve an accuracy of 99.69 %, 99.96 %, and 99.97 % for CICIDS2017, ToN-IoT Windows 10, and ToN-IoT Windows 7 respectively.

1. Introduction

Nowadays, it is not a secret how the Internet of Things is changing our interactions with the real world. IoT stands for the Internet of Things, a network of devices with sensors, software, and communication capability allowing them to collect and share data (Díaz et al., 2016). They can vary from home gadgets like smart thermostats and light bulbs to complex systems like monitoring systems and self-driving cars. The variety of IoT applications is vast: from enhancing energy efficiency at homes to improving security systems and optimizing manufacturing processes in industries (Asghari et al., 2019). Despite the fact that the benefits of the IoT are enormous, connectivity brings about some security issues. Resource-limited Internet of Things devices are vulnerable to an array of threats from data to device compromise. It should be emphasized that the protection of IoT is no longer a choice but a necessity that the real world should take into account.

To address the security issues arising from the devices connected to the Internet, different methods are employed to safeguard such systems (Chee et al., 2024; Issa et al., 2023; Ninu, 2023; Zhang et al., 2023). First

of all, data encryption during transmission and storage is needed to keep the confidentiality of the information that is shared between devices and servers. Moreover, device and user authentication is key to ensuring that data and functions are given only to authorized entities. Furthermore, the use of firewalls and intrusion prevention tools is essential for protection against attacks, which are based on network monitoring, behavior spotting, and intrusion attempt thwarting. An intrusion detection system (IDS) is a key component of security tools. These systems on a continuous basis try to spot the traffic patterns and the device actions which are the signs of being hacked. Using techniques of log analysis, flow monitoring, and anomaly detection, intrusion detection systems are capable of spotting risks quickly. They provide proactive monitoring and fast reaction in the IoT ecosystem with their layered configurations of remote sensors, centralized analyzers, and management interfaces (Liao et al., 2013).

In the area of intrusion detection, a range of techniques are used, each having its own benefits and drawbacks. Signature-based detection (Díaz-Verdejo et al., 2022) uses an algorithm to match patterns in network traffic or system activity with a database containing

* Corresponding author.

E-mail addresses: sarra.cherfi@univ-jijel.dz (S. Cherfi), ammar.boulaiche@univ-jijel.dz (A. Boulaiche), ali.lemouari@univ-tam.dz (A. Lemouari), abdelhafid.abouaissa@uha.fr (A. Abouaissa).

<https://doi.org/10.1016/j.eswa.2025.130122>

Received 24 July 2025; Received in revised form 14 October 2025; Accepted 19 October 2025

Available online 23 October 2025

0957-4174/© 2025 Elsevier Ltd. All rights reserved, including those for text and data mining, AI training, and similar technologies.

attack signatures. Although they are very good at detecting threats, those systems may be inadequate to face polymorphic or new attacks. On the opposite side, rule-based detection (Liu et al., 2018) involves a series of rules or specifications that define the behavior indicating any deviations from these norms as an intrusion. Consequently, it is less effective as it requires a lot of human intervention and it has a high rate of false positives. Unlike the behavior-based detection (Roy et al., 2022) used in our work which embraces analysis of device and user behaviors over time to report anomalies on established patterns that might indicate intent. Thanks to the implementation of machine learning and statistical analysis, behavior-based systems are able to adapt to any new threats and avoid false alarms, while allowing for adaptability and inclusion of new risks, all within a complex IoT environment.

Several approaches have been applied in intrusion detection systems (IDS), ranging from traditional machine learning to various deep learning techniques (Ghaffari et al., 2024; Liao et al., 2024). These approaches have demonstrated significant capabilities in automating data analysis, adapting to new intrusion patterns, and improving detection accuracy. However, their effectiveness is often constrained by the complexity and redundancy of large-scale network datasets. To address this, our study first applies the K-means clustering algorithm for data reduction, aiming to minimize computational overhead while retaining the essential structure and discriminative features of the data. This preprocessing step enhances model efficiency and reduces training time. After this data optimization stage, a convolutional neural network (CNN) (Mohammadpour et al., 2022) is employed to learn and classify intrusion behaviors. Among the deep learning architectures explored for IDS, such as recurrent neural networks (RNNs), autoencoders, generative adversarial networks (GANs), and transformer-based models, CNNs provide several practical advantages. They excel at capturing local spatial dependencies in traffic features, require fewer computational resources than transformers, and are more stable to train compared to RNN-based models that often suffer from vanishing gradients. According to a systematic review by Aversano et al. (2021), CNNs are the most widely used deep learning approach for intrusion detection in IoT environments, being adopted in more than 32% of studies, compared to about 17% for RNNs and 13% for deep autoencoders. The remaining research is distributed among various other architectures. This dominance reflects CNNs' strong ability to automatically extract relevant and hierarchical features from complex network data, making them especially effective for intrusion detection tasks. Moreover, the integration of K-means clustering with CNN further distinguishes our approach from existing hybrid methods. Whereas previous studies typically used clustering solely for preliminary feature selection or post-hoc grouping, our framework establishes a stronger interaction between data structuring and feature learning. This design enables the CNN to focus on the most informative and representative data regions, improving both training efficiency and detection accuracy. Consequently, the proposed CNN-K-means hybrid offers a balanced solution that combines the efficiency of data compression with the precision of deep learning for intelligent and adaptive intrusion detection.

The development of the intrusion detection system was based on two publicly available datasets: CICIDS2017 and ToN-IoT. CICIDS2017 was employed to build a model for identifying network attacks (NIDS: Network Intrusion Detection System) offering a range of simulated attacks that can be injected into the network to represent attack scenarios faced by systems. To complete our model, we have exploited the Windows version of the ToN-IoT dataset to detect host-based attacks (HIDS: Host Intrusion Detection System). ToN-IoT includes attacks aimed directly at devices providing an in-depth insight into potential threats targeting IoT systems at the host level. By merging these two datasets, we have developed an adaptable intrusion detection system to effectively detect and respond to both network-based and host-based threats in IoT environments. This approach has allowed us to deliver security against an array of attacks bolstering the protection of IoT systems, against emerging and

evolving threats. The key contributions of this paper are mentioned below.

- First, we cleaned and prepared the data from the two datasets used (CICIDS2017 and ToN-IoT) by eliminating fuzzy values, coding categorical data, normalizing the data, and then selecting a set of the most significant attributes using the Pearson correlation coefficient.
- Then, these pre-processed datasets were divided into two parts, one for testing and the other for training. The latter was then fed into the K-means algorithm to reduce the size of the training set while preserving the essential features of the data.
- Next, we used these training subsets resulting from the application of the K-means algorithm to train our convolutional neural network.
- Finally, we assessed our model by calculating metrics that permitted us to prove its efficiency in detecting anomalies in the IoT.

The rest of the paper is organized as follows. Section 2 presents some recent studies on intrusion detection in the Internet of Things domain. Section 3 describes our proposed intrusion detection model, the datasets used for training and testing our model, the preprocessing techniques applied to these datasets, and the size reduction method used in our work on the training set, before concluding with the tools used for training and testing. In Section 4, the experimental procedure and results are briefly discussed, and a comparison with previous research is made. Finally, Section 5 concludes the study and offers some suggestions for future work.

2. Related work

Intrusion detection in IoT systems is now becoming more and more of a topic of discussion. This has been mainly influenced by the many IoT devices that are now being used and the growing sophistication of cyber attacks. This part of the paper includes some recent research on intrusion detection in the IoT domain, where different types of machine learning and deep learning algorithms are employed.

Tang et al. (2020) suggested the implementation of a LightGBM-based IDS and auto-encoder to resolve the issues presented by the evasive nature of intrusion behaviors and the enormous amount of data in the era of big data. Their LightGBM-AE model includes three steps: data preprocessing, feature selection, and classification. The model relies on the LightGBM algorithm for feature selection, which is further trained and detected using an autoencoder. The intrusion detection method focuses on the reconstruction error that takes place between the input data and the reconstructed data in a way that allows the differentiation between the normal and abnormal. Running the experiment on the NSL-KDD dataset revealed that this procedure is quite successful in distinguishing attacks from normal activities, so it surpasses other well-known methods, including Decision Tree, Random Forest, KNN, GBDT, and XGBoost in terms of accuracy, recall, and F1-score.

Using various machine learning techniques, Fraihat et al. (2023), Vishwakarma and Kesswani (2023), and Samantaray et al. (2024) have contributed to the evolution of IoT IDSs. Hence Fraihat et al. suggested a Network Intrusion Detection System (NIDS) that employs a modified Arithmetic Optimization Algorithm (AOA) for feature selection to achieve a drastically reduced number of features. This optimization resulted in a very accurate detection model with high accuracy in both binary classification and multi-classification tasks depending on the datasets NF-CSE-CIC-IDS2018-v2, NF-ToN-IoT-v2, NF-UNSW-NB15-v2, and NF-BoT-IoT-v2. Vishwakarma et al. designed a two-phase IDS: The first phase is variations of the Naive Bayes classifier, and the second is the use of an unsupervised elliptic envelope for benign data analysis. This approach produced high accuracy rates on several datasets including NSL-KDD and CIC-IDS2017. Samantaray et al. addressed the problem of high-dimensional data using the MaxAbsScaler algorithm for feature scaling to improve multiclass classification accuracy through machine learning approaches such as SVM, KNN, and Random Forest. These studies show that feature selection and scaling are important in

improving the effectiveness of IDS for IoT environments. Nevertheless, practical ML techniques usually require sturdy feature engineering and capture only the shallow connections between data, which limits their learning capacity. Moreover, their detection efficiency degrades when used for highly dimensional IoT data and cannot satisfy the growing complexity and the need for IoT data types.

Moving on to work using deep learning methods, which offer the advantage of handling complex data models and achieving greater accuracy compared with traditional machine learning methods, [Nandanwar and Katarya \(2024\)](#) and [Ahmad et al. \(2023\)](#) made significant contributions. In both these works CNN-GRU model is used. Nandanwar et al. introduced AttackNet to identify botnet attacks in industrial IoT networks (IIoT) using deep learning techniques in an adaptive CNN-GRU model. This model produced very compelling test accuracies as well as precision and recall scores that were significantly higher compared to the current methods for the N_BaIoT dataset. Ahmad et al. also proposed a hybrid model with the integration of CNN with GRU algorithms and Slime Mould Algorithm (SMA) for optimization. In the case of the KDD dataset, the proposed model was able to detect DDoS attacks with high accuracy. Similarly, another recent study by [Javeed et al. \(2023\)](#) developed a novel deep learning-based intrusion detection system for edge-enabled smart agriculture operating in extreme environments. Their hybrid approach combined bidirectional gated recurrent units (Bi-GRU) and long short-term memory (LSTM) networks with a softmax classifier to effectively detect edge-level attacks. To accelerate the training process and handle lengthy network sequences, the model employed truncated backpropagation through time (TBPTT).

More recently, researchers have turned to transformer-based architectures for intrusion detection, given their strong ability to model long-range dependencies and contextual relationships in network traffic. The authors in [Nadiah et al. \(2025\)](#) proposed DistillGuard, a transformer-based IDS for IoT environments that employs a hybrid attention mechanism combining multi-head self-attention and cross-attention layers to capture temporal and contextual dependencies. It further uses selective gradient-based knowledge distillation (SG-KD) to transfer knowledge from a transformer teacher model to a lightweight student model, improving efficiency and accuracy while maintaining interpretability through gradient-based visualization. Likewise, [Manocchio et al. \(2024\)](#) developed FlowTransformer, a flexible transformer framework for NIDSs that models long-term network behaviors and allows modular substitution of components such as input encoding, transformer backbone, and classification head. Evaluations on benchmark datasets using GPT-2 and BERT architectures showed that optimized component configurations can reduce model size and inference time without compromising accuracy.

Continuing with studies that have employed hybrid deep learning architectures, the approaches taken in [Altunay and Albayrak \(2023\)](#), [Anitha et al. \(2023\)](#), [Du et al. \(2023\)](#), [Lilhore et al. \(2023\)](#) leveraged combinations of CNN and LSTM networks to capture both spatial and temporal dependencies in the data. This structure enables the models to learn complex associations in network traffic, thereby improving robustness and detection accuracy. Using a deep neural network, the authors of [Lu et al. \(2024\)](#) proposed a novel IIoT intrusion detection framework that addresses challenges such as extreme class imbalance, redundant features, and the limitations of traditional IDS techniques. They employed hierarchical clustering for under-sampling, optimal feature selection to remove redundancy, and parallel subnetworks for deep learning-based intrusion detection, achieving significantly improved performance. In addition, [Ahmad et al. \(2024\)](#) introduced a deep transfer learning-based IDS for IoT and IIoT environments. Their approach utilized the Edge-IIoT dataset, transforming network data into image representations for CNN-based processing, followed by hyperparameter optimization through a random search algorithm. The best-performing models were then integrated using an ensemble technique to enhance detection accuracy and generalizability across 14 attack types. Moreover, applying quantization to the top models reduced resource

consumption without compromising accuracy, demonstrating the practicality of the proposed method for resource-constrained IoT systems.

In addition to deep learning and transfer learning approaches, researchers have also explored deep reinforcement Learning (DRL) and federated learning (FL) to enhance intrusion detection in dynamic and distributed environments. The authors in [Mesadieu et al. \(2024\)](#) proposed a DRL-based anomaly detection framework for supervisory control and data acquisition (SCADA) systems. Their model employs a Q-network architecture to learn complex attack patterns and improve the adaptability of detection mechanisms in industrial control systems (ICS). The framework was validated on the WUSTL-IIoT-2018 and WUSTL-IIoT-2021 datasets, demonstrating its effectiveness in identifying sophisticated cyber threats and improving the resilience of critical infrastructure. Similarly, [Kaur \(2024\)](#) developed a federated learning (FL)-based intrusion detection model for industrial IoT (IIoT) networks that integrates GRUs for temporal feature extraction during decentralized FL training. To enhance model aggregation and device selection, a DRL mechanism was incorporated to identify IIoT nodes with high-quality data while maintaining energy efficiency and data privacy. Experimental validation on recent IIoT datasets confirmed that this hybrid FL-DRL framework provides improved detection performance and robustness compared with conventional FL-based IDS methods.

Despite the promising results demonstrated in the aforementioned studies, challenges persist in detecting anomalies in IoT networks, particularly with regard to performance, scalability, and computational efficiency. The application of datasets in the realm of machine learning has become commonplace presenting challenges such as prolonged computational time and resource demands for model training. Moreover, utilizing datasets can result in overfitting, where the model fixates on training data details at the expense of essential general characteristics for effective generalization. An additional hurdle is the class imbalance, where certain classes are disproportionately represented compared to others creating issues for machine learning algorithms that tend to prioritize majority classes while neglecting minority ones. To tackle these issues, we suggest a method centered around employing the K-means machine learning algorithm to downsize the training dataset, thereby expediting the training process, enhancing generalization accuracy, improving efficiency, and ensuring consideration of minority classes. The algorithm serves to select a subset of training data while maintaining representation, across all classes facilitating effective model training.

3. The proposed deep learning-based model

In this section, the intrusion detection model is presented in detail, starting with the introduction of the datasets used and the necessary transformations to prepare them to be interpreted by our model, including data cleaning, digitization, standardization, and attribute selection. Then, the importance of the K-means algorithm in selecting the training data is highlighted, allowing for complexity reduction while preserving data representativeness. The section concludes by presenting the deep learning method used to train our model, which is the CNN. Additionally, the metrics used to evaluate the model's performance are discussed, thus providing a comprehensive assessment of its effectiveness in the detection task. [Fig. 1](#) gives an overview of the approach used in our work.

3.1. Datasets description

In this study, two public datasets, CICIDS2017 and ToN-IoT, were utilized to investigate IoT attack detection. These datasets were selected for their relevance and comprehensive coverage of various IoT attack scenarios, which remain critical for our analysis and conclusions.

CICIDS2017 dataset: The dataset created by the Canadian Cybersecurity Institute (CIC) ([Sharafaldin et al., 2018](#)) is a resource for developing systems that can detect activities on networks. It contains information about network events spanning days presenting a variety of attack scenarios. This dataset consists of 79 features and 15 categories with

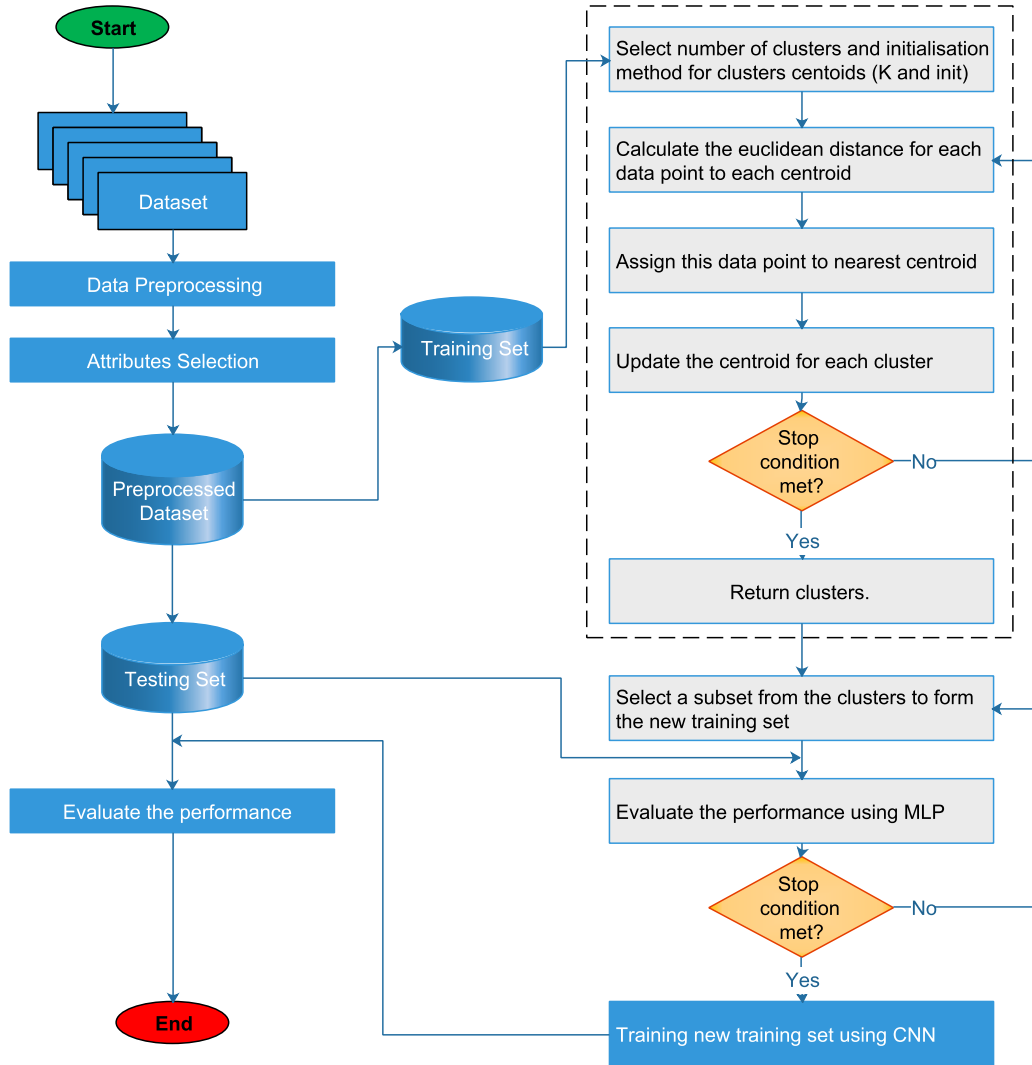


Fig. 1. Overview of the proposed deep learning-based model.

Table 1
Class distribution in CICIDS2017 dataset.

N°	Traffic class	Count of samples
0	Benign	2 273 097
1	Bot	1966
2	DDoS	128 027
3	Port Scan	158 930
4	Infiltration	36
5	Brute Force	1507
6	XSS	652
7	SQL Injection	21
8	FTP Patator	7938
9	SSH Patator	5897
10	DoS Slowloris	5796
11	DoS Slowhttptest	5499
12	DoS Hulk	231 073
13	DoS GoldenEye	10 293
14	Heartbleed	11

one representing benign network traffic and the rest indicating types of known anomalies. As the dataset includes more examples, features, and classifications, it becomes increasingly useful for analyzing network traffic behavior. Table 1 illustrates the distribution of all categories, within this dataset.

ToN-IoT dataset: This dataset was synthesized from a vast network that was jointly developed by the University of New South Wales (UNSW) and the Australian Defence Force Academy (ADFA) (Moustafa, 2021). This infrastructure included different elements like physical systems, cloud platforms, virtual devices, and IoT sensors. The recordings are made by several devices that take various angles of the network from IoT/IIoT, networking, Linux, and Windows. For our model training, we decided to use Windows systems data. This dataset includes two versions, one for Windows 7 and the other one for Windows 10. Each is presented as a CSV file that is tailored to measure the efficiency of machine learning methods. The datasets used contain 134 and 126 features and involve normal traffic and nine different categories of attack. The distribution of classes of these datasets is detailed in Tables 2 and 3 for ToN-IoT Windows 10 and ToN-IoT Windows 7 respectively.

3.2. Datasets preprocessing

The initial phase of data preparation is crucial for building precise machine learning and deep learning models. When data is collected from sources, it often contains errors like noise, missing details, and inconsistencies making it unsuitable for analysis by machine learning programs. The goal of the processing stage is to fix these issues through a series of corrective actions.

Table 2
Class distribution in ToN-IoT windows 10 dataset.

N°	Traffic class	Count of samples
0	Normal	9740
1	DDoS	4539
2	Password	3594
3	XSS	1240
4	Injection	606
5	DoS	505
6	Scanning	434
7	MITM	13

Table 3
Class distribution in ToN-IoT windows 7 dataset.

N°	Traffic class	Count of samples
0	Normal	9874
1	DDoS	2126
2	Backdoor	1753
3	Injection	987
4	Password	755
5	Scanning	225
6	Ransomware	72
7	XSS	4

In our case, the data processing phase began with a careful examination of the dataset to identify and correct any missing or inaccurate data points. This involved eliminating data elements like NaN (Not a Number) and infinite values as well as getting rid of redundant rows. The next step was the digitization of data, a necessary process because two techniques were used, K-means and CNN, which operate only on numerical inputs. Since some variables were categorical in nature, it was necessary to convert them into form for processing. For this purpose, the one-hot encoding method was applied, converting categorical variables into binary vectors. This approach maintained the meanings while enabling their incorporation, into machine learning systems.

Once the data had been prepared and encoded, our next step was to normalize the data. This step is crucial to ensure that all features are treated fairly in machine learning models. Normalization includes adjusting features to obtain a mean between zero and one standard deviation, in order to guarantee data consistency and account for variations in the feature scale. This process guarantees top-notch performance and efficiency with methods. The data were standardized using Eq. (1).

$$X_{std} = \frac{X - \text{mean}(X)}{\text{std}(X)} \quad (1)$$

Where :

- X : represents the variable to be standardized.
- $\text{mean}(X)$: is its mean.
- $\text{std}(X)$: is its standard deviation.

The data preparation phase concludes with a crucial task necessary for generating a reliable and coherent detection model, which is attribute selection. In this step, Pearson's correlation coefficient is used to assess the relationships between the various attributes. After calculating this coefficient for each attribute relative to the target attribute using Eq. (2), we select a set of attributes with a correlation coefficient greater than a fixed threshold. This approach allows us to retain only the attributes with a strong relationship with the target attribute, ensuring the relevance of the variables incorporated into our model. The Pearson correlation coefficient is calculated using the following formula, where r represents the correlation coefficient, X_i and Y_i denote the individual

data points, and $\bar{X}$ and $\bar{Y}$ signify the respective means of the variables.

$$r = \frac{\sum_{i=1}^n (X_i - \bar{X})(Y_i - \bar{Y})}{\sqrt{\sum_{i=1}^n (X_i - \bar{X})^2} \sqrt{\sum_{i=1}^n (Y_i - \bar{Y})^2}} \quad (2)$$

3.3. K-means clustering algorithm

This section focuses on the use of the K-means machine learning algorithm to downsample the training set and keep its performance on the test set. Based on the above discussions, the motivation for this exploration arises from the problems associated with datasets, for instance, computation time and over-fitting. Class imbalance is another problem, which means that certain classes are underrepresented and therefore may negatively affect the model's ability to generalize accurately. In our research, we observe imbalanced class distributions, in the datasets as illustrated in Tables 1–3. To address these issues, we propose an approach that focuses on data selection and class imbalance management. The K-means algorithm is applied to select a subset of the training dataset incorporating the class distribution.

Once the data preparation phase had been completed, the process moved on to the training data selection phase. After splitting the dataset into two parts, one for training (75 %) and the other for testing (25 %), we applied K-means to the training data, setting the number of clusters according to the number of classes in the dataset. The K-means algorithm used in our case is described in Algorithm 1. Then, several experiments were performed to find the best data rate that would give us more accuracy with fewer false alarms.

Algorithm 1 K-means clustering algorithm.

Require:

- dataset with n data points.
- K : number of clusters ($K = \text{number of classes}$).
- init: initialization method for clusters centroids ($\text{init} = \text{random}$ in our case).
- random_state = random seed for reproducibility.

Ensure:

- centroids: final centroid of each cluster.
 - labels: cluster labels for each data point.
- 1: randomly select k data points from the dataset as initial centroids.
 - 2: **while** (!convergence or $\text{nbr_iterations} < \text{max}$) **do**
 - 3: calculate the euclidean distance for each data point to each centroid.
 - 4: assign this data point to the nearest centroid.
 - 5: update the centroid for each cluster.
 - 6: check for convergence.
 - 7: $\text{nbr_iterations}++$
 - 8: **end while**
 - 9: **return** cluster labels for each data points.
-

To evaluate the performance of each subset of datasets in the preliminary experiments, an MLP was employed. The choice of MLP is motivated by its relatively simple architecture, which makes it straightforward to implement and train, while still being capable of modeling non-linear relationships. Compared to more sophisticated neural network architectures, MLPs involve fewer trainable parameters, which reduces computational cost and training time, making them suitable as a baseline classifier.

The implemented MLP consists of an input layer corresponding to the number of features in each dataset, a single hidden layer with 50 neurons activated by the ReLU function, and an output layer using the softmax activation function for classification. The model was trained for 1000 epochs, using the Adam optimizer and categorical cross-entropy as the loss function. After training, the model was evaluated on the corresponding test set to assess its performance on each dataset subset. The detailed configuration of the MLP is summarized in Table 4.

Table 4
Configuration of the MLP used in the preliminary experiments.

Parameter	Value
Hidden layers	1
Neurons per hidden layer	50
Activation function (hidden layer)	ReLU
Activation function (output layer)	Softmax
Optimizer	Adam
Loss function	Categorical Cross-Entropy
Batch size	5000
Epochs	1000

Table 5
Configuration of the CNN used in the experiments.

Parameter	Value
Convolutional layers	3
Filters per layer	16, 32, 64
Kernel size	3
Padding	Same
Pooling	Max-pooling (pool size = 2)
Activation function (hidden layers)	ReLU
Activation function (output layers)	Softmax
Flatten layer	Yes
Fully connected (Dense) layer	50 neurons
Optimizer	Adam
Loss function	Categorical Cross-Entropy

3.4. Model training and testing

A convolutional neural network (CNN) was selected to build the classification model. CNNs are deep learning architectures well known for their ability to automatically learn and extract hierarchical feature representations from input data. Originally developed for image recognition tasks such as object detection and image segmentation, CNNs have proven effective in recognizing complex spatial patterns and local dependencies within structured data. In the context of intrusion detection, CNNs can capture intricate relationships among network traffic features, enabling efficient discrimination between normal and malicious behaviors without requiring extensive manual feature engineering. Their robustness, scalability, and capacity to generalize from large datasets make them particularly suitable for cybersecurity applications.

A CNN architecture was designed specifically for handling sequential data. It is made up of three convolution layers, where each uses a max-pooling layer that comes right after it, in order to extract the most relevant features. The convolutional layers consist of 16 units (neurons), 32 units, and 64 units respectively, which are able to capture different scale patterns. Thus, the CNN model can learn from these multiple sets of information at diverse scales thereby forming hierarchical representations. In this way, various abstracted features can be identified in terms of patterns.

For our multiclass classification task, where each data instance has to be assigned to one of several categories, we used an output layer with softmax activation. This activation assigns probabilities to each possible class, making it easier for the model to make the final decision. The hidden layer after flattening (flatten) has 50 neurons to enable more complex learning of intermediate features. The loss function used is the categorical cross-entropy loss, adapted to multiclass classification problems. The detailed configuration of the CNN is summarized in Table 5, while the overall architecture is illustrated in Fig. 2.

Having completed the training phase of the CNN, the next important task involves evaluating the model's performance through the testing phase. The evaluation of deep learning (DL) models for classification tasks, including the one addressed in this study, often relies on metrics derived from a confusion matrix (Fig. 3). This tool functions as a tabular representation, detailing how often a model accurately assigns a data sample to its true class label. As the model strives to discern

the appropriate classification for each data instance, its predictions are recorded and juxtaposed with truth labels in the field. Subsequently, the confusion matrix facilitates the calculation of the correct and incorrect classifications made by the model, offering insight into its performance in different classes.

After obtaining the confusion matrix, several key metrics are calculated to assess the performance of our classification model. These measures are as follows:

- **Accuracy:** Proportion of correct predictions among all model predictions. It is calculated using Eq. (3).

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} * 100\% \quad (3)$$

- **Recall:** Model's ability to identify all occurrences of a given class. It is obtained by applying Eq. (4).

$$Recall = \frac{TP}{TP + FN} * 100\% \quad (4)$$

- **Precision:** Proportion of correct positive predictions among all positive predictions in the model. Precision is calculated by Eq. (5).

$$Precision = \frac{TP}{TP + FP} * 100\% \quad (5)$$

- **False alarm rate:** Proportion of negative occurrences incorrectly identified as positive by the model. It can be obtained by Eq. (6).

$$Precision = \frac{FP}{FP + TN} * 100\% \quad (6)$$

- **F1 Score (Harmonic mean):** Metric combining precision and recall in a single value between 0 and 1, providing a synthetic assessment of classification. It is calculated using Eq. (7)

$$F1 - Score = \frac{2 * Precision * Recall}{Precision + Recall} \quad (7)$$

3.5. Complexity analysis

This section analyzes the computational complexity of the proposed intrusion detection pipeline. The pipeline consists of data preprocessing, clustering, subsampling, model training, and inference. Understanding the complexity of each stage highlights the computational bottlenecks and indicates how the pipeline scales with the number of samples, features, and network parameters.

- Preprocessing operations including scaling, encoding, and correlation-based feature selection are $O(N \times d)$, where N is the number of samples and d the number of features.
- Clustering using KMeans requires $O(i \times k \times N \times d)$, where i is the number of iterations and k the number of clusters.
- Subsampling clustered data into training and testing partitions requires scanning all samples, giving $O(N)$.
- Training the MLP with one hidden layer of size h for E epochs has complexity $O(E \times N \times h)$.
- Training the CNN with a total of $filters$ kernels for E epochs requires $O(E \times N \times d \times filters)$.
- Inference on a test set of size M requires $O(M \times d \times h)$ for the MLP and $O(M \times d \times filters)$ for the CNN, which is negligible compared to training.

The overall computational complexity of the pipeline can be expressed as:

$$O(N \times d) + O(i \times k \times N \times d) + O(N) + O(E \times N \times h) + O(E \times N \times d \times filters)$$

By dropping the lower-order terms, such as $O(N)$, which are negligible compared to the other terms, the complexity simplifies to:

$$O(i \times k \times N \times d + E \times N \times (h + d \times filters))$$

This shows that the pipeline is dominated by the training phase, particularly the CNN model.

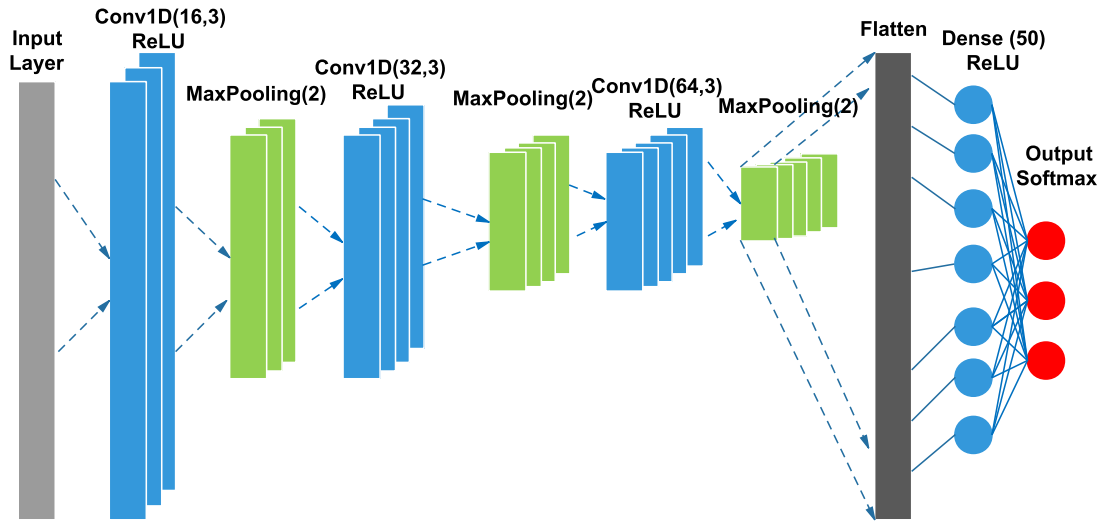


Fig. 2. Architecture of the CNN model used in the experiments.

Current class	Normal	TN	FP
	Attack	FN	TP
		Normal	Attack
		Predicted class	

Fig. 3. The confusion matrix.

Table 6
System setup and implementation details.

Specifications	Description
CPU	Intel® Core(TM) i5-4300M, 2.60GHz (dual-core)
Memory	8 GB RAM
Platform	Microsoft Windows 10
Programming Environment	Python version 3.9.12
Development Tool	Spyder IDE (version 5.1.5)
Main Libraries	Keras, Scikit-learn, Matplotlib, Pandas, NumPy

4. Experimental results and discussion

In this section, we present the experimental setup, including the environment and configurations used to implement our work, followed by the results obtained for the proposed intrusion detection model. The performance of the model is analyzed in detail, and comparisons are provided with other balancing strategies and recent approaches from the literature to highlight the effectiveness of our method.

4.1. System configuration

The experiments were conducted on a standard computing environment, with the main specifications summarized in Table 6. The implementation was carried out using Python and the Spyder development environment, supported by widely used libraries such as Keras and Scikit-learn for deep learning and machine learning tasks.

4.2. Results with K-means and MLP

To initiate the evaluation, we first investigated the performance of the proposed framework by integrating K-means clustering with a multi-layer perceptron (MLP) classifier. The purpose of this step is to assess

the effectiveness of K-means as a data balancing strategy and to identify the most suitable model configuration for subsequent experiments. The obtained results provide a preliminary benchmark and serve as the basis for selecting the best-performing model, which is then further refined and extended using a convolutional neural network (CNN).

4.2.1. The results of the CICIDS2017 dataset experiments

In this section, we examine the performance of the different classes in the CICIDS2017 dataset in terms of accuracy and false alarm rate, varying the data rate from 10 % to 100 %. These datasets are selected from the clusters resulting from the application of the K-Means algorithm, and the results obtained are presented in Tables 7 and 8. It is notable that the highest accuracy is achieved using only 10 % of the training data for several attack classes such as Bot, Brute Force, XSS, SQL Injection, DoS Hulk, and Heartbleed. However, only two classes, namely DDoS and SSH Patator, achieved this maximum value with the full dataset. When examining false alarm rates, similar trends are observed, with the lowest rates achieved using only 10 % of the data for classes such as Infiltration, Brute Force, XSS, SQL Injection, and Heartbleed. Notably, only the DDoS class recorded the lowest false alarm rate using the full training data set.

Table 9 presents the selected data rate for the training phase of our CNN for each class of the CICIDS2017 dataset, which is divided into 7 distinct files. Among them, the "Monday" file contains exclusively benign traffic, while the other files each contain a specific attack class along with normal traffic. For example, "Friday Bot" contains the Bot attack class, "Friday DDoS" includes DDoS attacks, "Friday PortScan" encompasses port scan attacks, "Thursday Web" includes web attacks such as Brute Force, XSS, and SQL Injection, "Tuesday" includes FTP Patator and SSH Patator, and "Wednesday" consists of DoS attacks.

For each file, we select the most appropriate training data rate to maximize overall accuracy while minimizing the false alert rate. It is worth noting that the entirety of the data is used only for 2 files, namely "Friday DDoS" and "Tuesday", while for three of them, only 10 % of the data is used. This approach has thus allowed us to optimize not only the accuracy but also the false alert rate and the calculation time.

4.2.2. The results of the ToN-IoT windows 10 dataset experiments

This section presents the results obtained for the ToN-IoT windows 10 dataset. Table 10 illustrates the variation in accuracy values for each data rate, while Table 11 presents the false alert rate. It is noteworthy that the maximum accuracy value for each attack class is observed at multiple data rates. For example, for the DDoS class, this value is achieved at rates of 90 % and 100 %, as is the

Table 7

Accuracy values for all attack classes in CICIDS2017 for the various experiments.

	10 %	20 %	30 %	40 %	50 %	60 %	70 %	80 %	90 %	100 %
Bot	99.87	99.80	99.76	99.77	99.70	99.70	99.71	99.69	99.65	99.64
DDoS	96.25	97.42	97.86	99.14	99.23	99.26	99.33	99.30	99.36	99.42
Infiltration	99.994	99.991	99.995	99.995	99.995	99.993	99.991	99.993	99.993	99.991
Port-scan	99.72	99.71	99.76	99.76	99.77	99.79	99.76	99.78	99.75	99.72
FTTP-Patator	99.88	99.94	99.96	99.99	99.98	99.99	99.98	99.99	99.98	99.99
SSH-Patator	99.79	99.81	99.83	99.81	99.81	99.82	99.84	99.84	99.84	99.85
Brute Force	99.71	99.53	99.44	99.34	99.07	98.03	99.06	99.03	98.97	98.96
XSS	99.91	99.81	99.74	99.71	99.69	99.62	99.68	99.67	99.61	99.59
SQL-Injection	99.99	99.98	99.99	99.99	99.99	99.99	99.98	99.99	99.98	99.98
DoS Slowloris	99.83	99.80	99.74	99.85	99.83	99.77	99.85	99.86	99.82	99.84
DoS Slowhttptest	99.85	99.86	99.88	99.89	99.89	99.88	99.90	99.88	99.88	99.88
DoS Hulk	96.61	94.52	93.01	91.96	92.18	92.67	92.98	93.29	93.35	93.79
DoS GoldenEye	99.79	99.84	99.88	99.89	99.91	99.92	99.93	99.91	99.93	99.93
Heartbleed	100	99.99	100	100	100	100	100	100	99.99	100

Table 8

False alarm rate values for all attack classes in CICIDS2017 for the various experiments.

	10 %	20 %	30 %	40 %	50 %	60 %	70 %	80 %	90 %	100 %
Bot	0.006	0	0.004	0.002	0	0.006	0.008	0.002	0.002	0.004
DDoS	6.10	4.52	4	1.65	1.52	1.48	1.37	1.46	1.38	1.28
Infiltration	0	0	0.002	0.002	0.001	0.001	0.002	0	0.002	0.004
Port-scan	0.06	0.08	0.06	0.07	0.05	0.04	0.06	0.05	0.06	0.54
FTTP-Patator	0.01	0.04	0.02	0.002	0	0.001	0.001	0.005	0.002	0.002
SSH-Patator	0.13	0.11	0.09	0.11	0.12	0.11	0.09	0.08	0.1	0.09
Brute Force	0	0	0.02	0.02	0.87	1.01	0.89	0.92	0.97	0.96
XSS	0	0	0	0.007	0.002	0	0	0.007	0.004	0
SQL-Injection	0	0	0	0	0	0	0	0	0	0
DoS Slowloris	0.05	0.01	0.01	0.05	0.06	0.13	0.02	0.02	0.07	0.05
DoS Slowhttptest	0.07	0.07	0.05	0.05	0.04	0.06	0.03	0.03	0.06	0.03
DoS Hulk	0.005	0.004	0.007	0.006	9.05	8.83	8.47	8.10	8.15	8.18
DoS GoldenEye	0.09	0.059	0.04	0.03	0.03	0.02	0.01	0.02	0.02	0.01
Heartbleed	0	0.0005	0	0	0	0	0	0.0005	0	0

Table 9

The selected models for CICIDS2017 dataset.

Files	Classes	SM	Accuracy (FD)	Accuracy (SM)	FAR (FD)	FAR (SM)
Friday Bot	Bot	10 %	99.64	99.87	0.04	0.006
Friday DDoS	DDoS	100 %	99.42	99.42	1.28	1.28
Friday PortScan	Port Scan	60 %	99.72	99.79	0.54	0.04
Thursday Infiltration	Infiltration	30 %	99.99	99.99	0.004	0.002
	Brute Force		98.96	99.71	0.96	0
Thursday Web	XSS	10 %	99.59	99.91	0	0
	SQL Injection		99.98	99.99	0	0
Tuesday	FTP Patator	100 %	99.99	99.99	0.002	0.002
	SSH Patator		99.85	99.85	0.09	0.09
	DoS Slowloris		99.84	99.83	0.05	0.05
	DoS Slowhttptest		99.88	99.85	0.03	0.07
Wednesday	DoS Hulk	10 %	93.79	96.61	8.18	0.005
	DoS GoldenEye		99.93	99.79	0.01	0.09
	Heartbleed		100	100	0	0

SM: Selected Model, FD: Full Dataset (without K-means), FAR: False Alarm Rate.

case for the false alarm rate. Upon analyzing these results, we observe that the accuracy and false alert rate values obtained for the entire dataset are similar or very close to those obtained with 90 % of the data.

Since this dataset is presented as a single file, the most appropriate data rate for all attack classes must be selected, taking into account both accuracy and false alert rates. Based on the results obtained and shown in Tables 10 and 11, the model using 90 % of the training data was selected, as the maximum accuracy value and the minimum false alert rate are achieved for almost all classes at this rate. Table 12 illustrates the difference between the results obtained for the initial model and the selected model, which means the model that used only 90 % of the training data to then train the CNN.

4.2.3. The results of the ToN-IoT windows 7 dataset experiments

Tables 13 and 14 present the results obtained for the ToN-IoT Windows 7 dataset in terms of accuracy and false alert rates for various experiments. It is noteworthy that the maximum accuracy value and the minimum false alert rate were observed at different data rates, ranging from only 10 % in the case of the DDoS class to 100 % for the DDoS, Password, and XSS classes. Additionally, for this dataset, we observe that 100 % accuracy is achieved for several classes at different data rates.

For this dataset, it is clear that the 100 % accuracy value is achieved for most classes when only 90 % of the training data is used. This is particularly notable for the Backdoor, DDoS, Injection, Ransomware, and Scanning classes, as illustrated in Table 15. As a result, we have opted for this model, which uses only 90 % of the data, to train our CNN.

Table 10

Accuracy values for all attack classes in ToN-IoT windows 10 for the various experiments.

	10 %	20 %	30 %	40 %	50 %	60 %	70 %	80 %	90 %	100 %
DDoS	98.60	99.09	98.58	98.54	99.69	99.51	99.86	99.63	99.90	99.90
DoS	99.24	100	99.92	99.86	99.98	100	99.98	100	100	100
Injection	99.96	99.98	100	99.98	100	100	100	100	100	100
MITM	99.76	99.86	99.88	99.88	99.84	99.90	99.94	99.92	99.94	99.86
Password	98.93	99.38	99.43	98.91	99.80	99.51	99.94	99.74	99.90	99.92
Scanning	99.51	99.49	99.53	99.41	99.80	99.86	99.88	99.84	99.94	99.92
XSS	99.94	99.98	99.96	99.96	99.98	100	100	99.96	100	100

Table 11

False alarm rate values for all attack classes in ToN-IoT windows 10 for the various experiments.

	10 %	20 %	30 %	40 %	50 %	60 %	70 %	80 %	90 %	100 %
DDoS	1.26	1.02	1.59	1.62	0.37	0.59	0.17	0.46	0.12	0.12
DoS	0	0	0	0	0	0	0	0	0	0
Injection	0	0	0	0	0	0	0	0	0	0
MITM	0	0	0	0	0	0	0	0	0	0
Password	1.30	0.44	0.33	0.58	0.10	0.02	0.02	0.07	0.02	0.02
Scanning	0.35	0.06	0	0	0.02	0	0	0	0	0
XSS	0.02	0.02	0	0.02	0	0	0	0	0	0

Table 12

The selected model for ToN-IoT windows 10 dataset.

Classes	Accuracy (FD)	Accuracy (SM)	FAR (FD)	FAR (SM)
DDoS	99.90	99.90	0.12	0.12
DoS	100	100	0	0
Injection	100	100	0	0
MITM	99.86	99.94	0	0
Password	99.92	99.90	0.02	0.02
Scanning	99.92	99.94	0	0
XSS	100	100	0	0

SM: Selected Model, FD: Full Dataset, FAR: False Alarm Rate.

4.3. Results with K-means and CNN

In this section, we evaluate the performance of our approach to reducing the size of the training dataset while preserving model performance by using the K-means technique for this task, followed by training a CNN.

The results obtained, shown in Table 16, are promising, demonstrating the effectiveness of our method in reducing dataset size while maintaining high model performance. The approach was evaluated on two distinct datasets: CICIDS2017 and ToN-IoT.

For the CICIDS2017 dataset, the model achieved an overall accuracy of 99.69 %, with recall, precision, and F1-score all at 99.36 %. Training was completed in 11.42 s per epoch, while inference required only 2 milliseconds per step, highlighting both the efficiency and effectiveness of the proposed method.

Similarly, for the ToN-IoT dataset, remarkable performance was observed. On the data subsets associated with Windows 10 and Windows 7, we achieved accuracy, precision, recall, and F1-score rates of 99.96 % and 99.97 % respectively. In addition, the model maintained low computational overhead, with training times of 186 ms/epoch and 156 ms/epoch, and testing consistently requiring just 2 ms/step.

These findings confirm that the integration of K-means for dataset size reduction with CNN for classification not only ensures robust and high-performing intrusion detection but also significantly reduces training and inference costs, making the approach well-suited for practical deployment in real-world environments.

After evaluating the overall performance of our approach, we examined in detail the results per attack class on the CICIDS2017 dataset. The

accuracy and false alarm rate values for each attack class are presented in Fig. 4. Analysis of the results reveals a high overall performance, with accuracy values ranging from 96.61 % to 100 % for all classes of attack. Attacks such as Infiltration, FTP Patator, SQL Injection, and Heartbleed achieved a maximum accuracy of around 100 %, indicating the exceptional ability of our model to detect these types of attacks with a very low margin of error. However, it should be noted that some attack classes perform slightly worse in terms of accuracy and false alarm rate. For example, the DoS Hulk class has an accuracy of just 96.61 %, suggesting that our model has difficulty distinguishing this attack from normal activity with a high degree of certainty. Similarly, the false alarm rate for the DoS Slowhttptest attack class is 0.08, suggesting a greater tendency to misidentify incidents of this attack class.

Despite these variations in performance between different attack classes, the high overall performance of our approach across all attack classes testifies to its ability to provide robust protection against a diverse range of threats. These results reinforce the credibility and applicability of our approach in real intrusion detection scenarios, where the ability to accurately identify attacks while minimizing false positives is essential to ensure the security of IT systems.

We move to the evaluation of performance by attack class on ToN-IoT datasets, where we distinguish two subsets that are ToN-IoT Windows 10 and ToN-IoT Windows 7. The results were shown in Fig. 5, whose first graph presents the results for the subset of Windows 10 while the second presents those of Windows 7, where each of these graphs highlighting the accuracy and false alarm rates for each attack class.

The results for ToN-IoT Windows 10 demonstrate impressive performance, with accuracy values reaching 100 % for most attack classes. Negligible false alarm rates are also observed, highlighting our model's ability to accurately detect a variety of attacks while minimizing false positives. This is particularly noteworthy for DoS, Injection, MITM, and XSS classes, where performance is perfect, indicating our model's exceptional reliability in detecting these attacks.

Similarly, results for ToN-IoT Windows 7 demonstrate outstanding performance, with accuracy values of 100 % for most attack classes, including Backdoor, DDoS, Injection, Password, Scanning, and XSS. In addition, false alarm rates continue to be extremely low, giving us greater confidence that our model can accurately distinguish between benign and malicious activity. Notably, the Ransomware attack class performed flawlessly, proving the effectiveness of our methodology in identifying complex threats such as ransomware.

Table 13

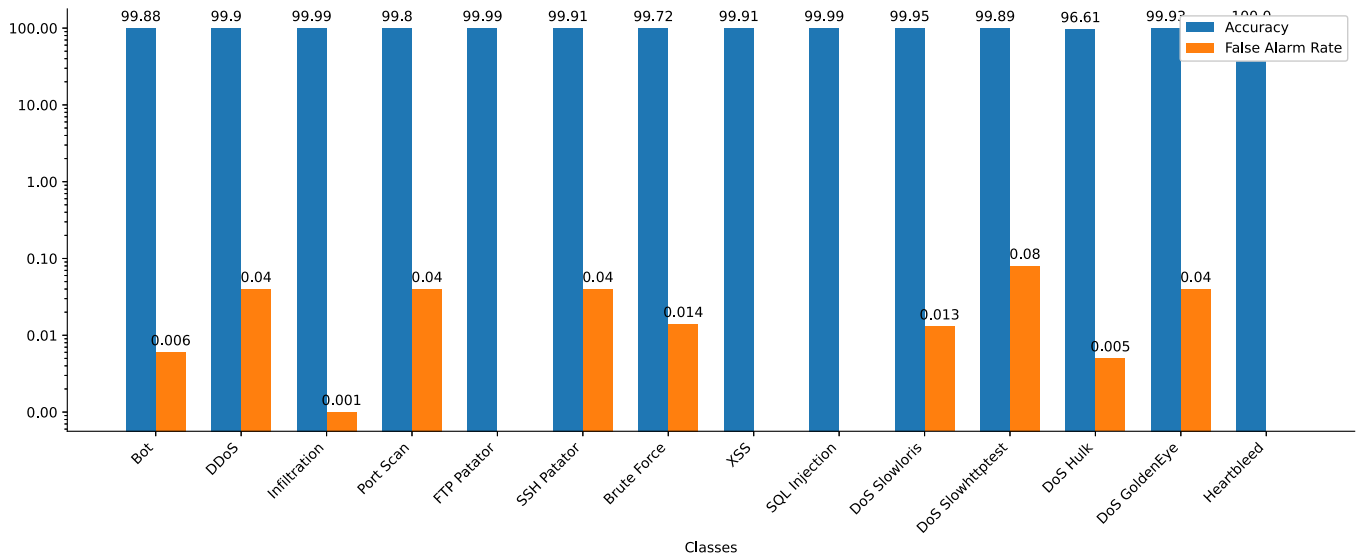
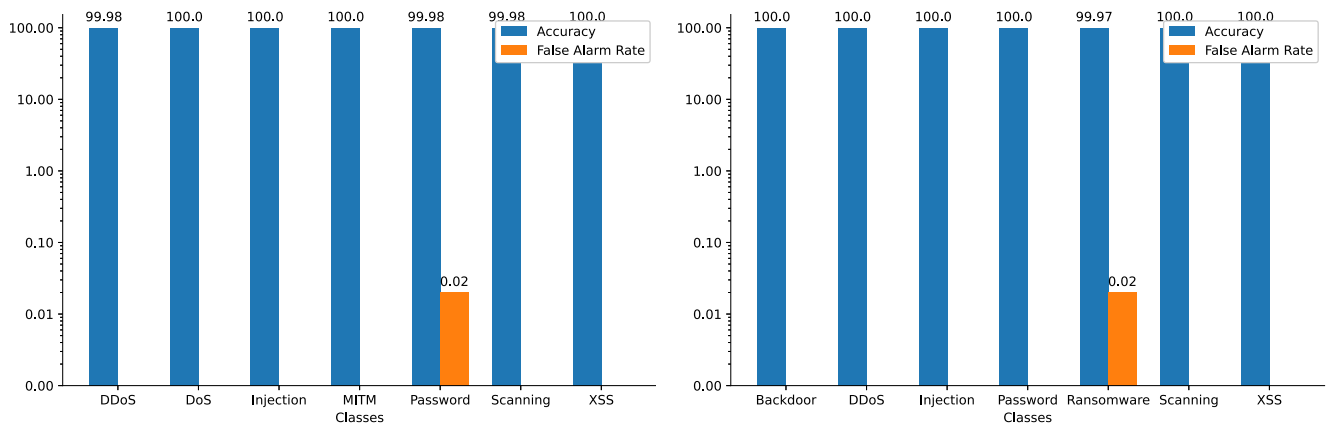
Accuracy values for all attack classes in ToN-IoT windows 7 for the various experiments.

	10 %	20 %	30 %	40 %	50 %	60 %	70 %	80 %	90 %	100 %
Backdoor	99.89	99.82	99.94	99.84	99.84	99.94	100	100	100	99.97
DDoS	100	99.94	100	100	100	100	100	100	100	100
Injection	99.31	98.91	99.92	99.97	99.87	99.87	99.92	100	100	99.94
Password	99.92	99.79	99.92	99.77	99.87	99.89	99.94	99.94	99.94	99.94
Ransomware	99.79	100	99.92	100	99.97	99.89	100	100	100	99.97
Scanning	99.54	99.11	98.37	98.78	99.62	99.94	99.74	99.94	100	99.67
XSS	99.94	99.92	99.92	99.97	99.97	99.97	99.94	99.94	99.94	99.97

Table 14

False alarm rate values for all attack classes in ToN-IoT windows 7 for the various experiments.

	10 %	20 %	30 %	40 %	50 %	60 %	70 %	80 %	90 %	100 %
Backdoor	0.1	0.18	0.05	0.16	0.16	0.05	0	0	0	0.02
DDoS	0	0.05	0	0	0	0	0	0	0	0
Injection	0.3	0.1	0.02	0.02	0.13	0.13	0.08	0	0	0.05
Password	0.08	0.08	0.08	0.02	0.02	0.02	0.05	0.05	0.05	0.02
Ransomware	0	0	0	0	0	0.07	0	0	0	0.02
Scanning	0.02	0	0	0	0	0	0	0	0	0
XSS	0	0	0	0	0	0	0	0	0	0

**Fig. 4.** Accuracy and false alarm rate for each attack class in the CICIDS2017 dataset.

(a) ToN-IoT Windows 10 dataset.

(b) ToN-IoT Windows 7 dataset.

Fig. 5. Accuracy and false alarm rate for each attack class in the ToN-IoT dataset.

Table 15

The selected model for ToN-IoT windows 7 dataset.

Classes	Accuracy (FD)	Accuracy (SM)	FAR (FD)	FAR (SM)
Backdoor	99.97	100	0.02	0
DDoS	100	100	0	0
Injection	99.94	100	0.05	0
Password	99.94	99.94	0.02	0.05
Ransomware	99.97	100	0.02	0
Scanning	99.67	100	0	0
XSS	99.97	99.94	0	0

SM: Selected Model, FD: Full Dataset, FAR: False Alarm Rate.

4.4. Comparative study

In order to evaluate the effectiveness of the proposed CNN-K-means model, we compared its performance with three other variants of CNN: the baseline CNN model without any imbalance-handling technique, CNN combined with SMOTE oversampling, and CNN trained with a cost-sensitive strategy. The comparison was conducted across the three datasets used, and the results are summarized in Table 17.

From the table, it can be observed that the proposed CNN-K-means approach consistently outperforms the baseline CNN and the other imbalance-handling techniques in terms of accuracy, recall, precision, and F1-score, while also achieving significantly reduced training time in most cases. For example, on the CICIDS2017 dataset, CNN-K-means achieved an accuracy of 99.69%, surpassing the baseline CNN (99.01%), SMOTE-based CNN (98.71%), and cost-sensitive CNN (97.59%). Similarly, on the ToN-IoT datasets, CNN-K-means not only maintained comparable or superior classification performance but also

reduced the computational overhead, demonstrating its efficiency and scalability.

To provide a more detailed analysis, Table 18 reports the class-wise performance of the proposed CNN-K-means model compared with the baseline CNN, cost-sensitive CNN, and SMOTE-based CNN on the CICIDS2017 and ToN-IoT datasets. The results show that CNN-K-means consistently achieves either the best or near-best accuracy across most attack categories. For example, on the CICIDS2017 dataset, CNN-K-means attains superior detection rates for challenging classes such as Bot (99.88%), XSS (99.91%), and DoS Hulk (96.61%), outperforming the baseline and other balancing strategies. Similarly, on the ToN-IoT datasets, CNN-K-means achieves perfect classification (100%) for several critical attack types, including Injection, MITM, and Backdoor, while maintaining competitive performance on all other classes.

Finally, to validate the effectiveness of the proposed approach, we conducted a comparative analysis against several recent works from the literature, focusing on model accuracy. The comparison was performed on the CICIDS2017 and ToN-IoT datasets under both Windows 10 and Windows 7 environments, and the results are summarized in Table 19.

Our approach, based on the use of K-means followed by CNN, demonstrated comparable or even superior performance to the other methods studied. On the CICIDS2017 dataset, our approach achieved an accuracy of 99.69%, slightly outperforming previous work such as Nguyen and Le (2023) with 99.27% and Wang et al. (2023) with 99.15%. For the ToN-IoT dataset, our results for Windows 10 and Windows 7, 99.96% and 99.97% respectively, significantly outperformed those of previous work, including Tareq et al. (2022) and Khan et al. (2023).

In conclusion, the results obtained validate the effectiveness of our approach in reducing the size of the training dataset while preserving model performance.

Table 16

Results of our proposed model on the different datasets used.

Dataset	Accuracy	Recall	Precision	F1-Score	Training Time	Testing Time
CICIDS2017	99.69	99.36	99.36	99.36	11.42s/epoch	2ms/step
ToN-IoT windows10	99.96	99.96	99.96	99.96	186ms/epoch	2ms/step
ToN-IoT windows7	99.97	99.97	99.97	99.97	156ms/epoch	2ms/step

Table 17

Performance comparison of CNN-K-means with other imbalance-handling techniques.

Dataset	Balancing Technique	Accuracy	Recall	Precision	F1-Score	Training Time
CICIDS2017	Baseline (CNN)	99.01	98.91	99.14	99.02	25s/epoch
	With SMOTE	98.71	98.70	98.72	98.71	66s/epoch
	With cost-sensitive	97.59	97.59	97.60	97.59	25s/epoch
	With K-means	99.69	99.36	99.36	99.36	11.42s/epoch
ToN-IoT windows10	Baseline (CNN)	99.76	99.74	99.76	99.75	186ms/epoch
	With SMOTE	99.82	99.82	99.82	99.82	690ms/epoch
	With cost-sensitive	99.84	99.84	99.84	99.84	186ms/epoch
	With K-means	99.96	99.96	99.96	99.96	186ms/epoch
ToN-IoT windows7	Baseline (CNN)	99.97	99.97	99.97	99.97	218ms/epoch
	With SMOTE	99.94	99.94	99.94	99.94	800ms/epoch
	With cost-sensitive	99.94	99.94	99.94	99.94	218ms/epoch
	With K-means	99.97	99.97	99.97	99.97	156ms/epoch

Table 18
Class-wise performance comparison of CNN–K-means with other imbalance-handling techniques.

Dataset	Classe	Baseline(CNN)	Cost-sensitive	SMOTE	K-means
CICIDS2017	Bot	99.62	97.21	98.88	99.88
	DDoS	99.42	99.42	99.41	99.9
	Infiltration	99.99	96.55	99.99	99.99
	Port Scan	99.78	99.78	99.78	99.80
	FTP Patator	99.99	99.98	99.93	99.99
	SSH Patator	99.89	99.31	99.83	99.91
	Brute Force	99.05	98.98	99.16	99.72
	XSS	99.59	97.44	98.25	99.91
	SQL Injection	99.98	99.41	99.72	99.99
	DoS Slowloris	99.93	99.86	99.91	99.95
	DoS Slowhttptest	99.93	99.84	99.94	99.89
	DoS Hulk	95.51	94.59	95.37	96.61
	Dos GoldenEye	99.96	99.91	99.96	99.93
	Heartbleed	100	99.99	100	100
ToN-IoT windows10	DDoS	99.92	100	99.98	99.98
	DoS	99.92	99.98	99.98	100
	Injection	99.98	100	100	100
	MITM	99.92	99.94	99.96	100
	Password	99.96	99.96	99.98	99.98
	Scanning	99.88	99.86	99.86	99.98
ToN-IoT windows7	XSS	100	100	100	100
	Backdoor	100	100	100	100
	DDoS	99.97	100	100	100
	Injection	100	99.97	99.97	100
	Password	99.97	100	100	100
	Ransomware	100	99.97	99.97	9.97
	Scanning	100	100	100	100
	XSS	100	100	100	100

Table 19
Performance comparison of the proposed intrusion detection model with recent IDSs.

Dataset	Article	Year	Method	F1-Accuracy
CICIDS2017	Nguyen and Le (2023)	2023	Multistaged robust decision trees - SAINT	99.27 %
	Wang et al. (2023)	2023	ResNet + TranBiLSTM	99.15 %
	Lazzarini et al. (2023)	2023	DIS-IoT	98.70 %
	Zhang et al. (2024)	2024	GRU-CNN	97.33 %
	Osa et al. (2024)	2024	DNN	99.68 %
	Kumar et al. (2024)	2024	DCRNN	99.56 %
	Proposed approach	2025	K-means-CNN	99.69 %
	Tareq et al. (2022)	2022	DenseNet	97.87 %
ToN-IoT windows 10	Mohamed et al. (2022)	2022	Inception Time	98.30 %
			Weighted KNN	98.22 %
	Khan et al. (2023)	2023	Medium Neural Network	98.39 %
			RNN-GRU	84 %
			K-means-CNN	99.96 %
ToN-IoT windows 7	Tareq et al. (2022)	2022	DenseNet	98.36 %
			Inception Time	99.21 %
	Khan et al. (2023)	2023	RNN-GRU	88 %
			K-means-CNN	99.97 %

These findings open up new possibilities for prospective applications in a variety of industries where data is expensive or hard to collect.

5. Conclusion

In this paper, we present an intrusion detection model for the IoT network and devices (NIDS + HIDS) based on deep learning using a CNN for training, which is excellent at capturing spatial dependencies in data, making it well suited to tasks such as image recognition and, in our case, feature extraction from network traffic data. However, before this stage, we introduced the concept of reducing the size of the training dataset by employing the K-means algorithm, which is a clustering algorithm commonly used for data segmentation and centroid initialization. This approach significantly reduced computation time and addressed the problems of over-fitting and class imbalance. The results obtained demonstrated the effectiveness of our model, achieving accuracy rates of 99.69 %, 99.96 %, and 99.97 % for the CICIDS2017, ToN-IoT Windows 10, and ToN-IoT Windows 7 datasets, respectively.

This study assumes that the benchmark datasets used are representative of real-world IoT network traffic and that the selected features are sufficient to distinguish between normal and malicious behaviors. The K-means algorithm assumes that data clusters are approximately spherical and well-separated, which may not accurately reflect the complex distribution of real network traffic. In addition, the proposed model relies on static feature representations and does not incorporate protocol-level or contextual network information that may further enhance detection accuracy. Experimental evaluations were conducted on benchmark datasets only, and the real-time scalability and resource efficiency of the model on constrained IoT hardware have not yet been validated. Future research will focus on enhancing feature representation through advanced scaling and dimensionality reduction techniques, such as autoencoder-based compression and principal component analysis, to further improve model efficiency. Transfer learning will be explored to adapt pretrained CNN models to specific IoT environments, thereby improving generalization across diverse device types.

and network conditions. In addition, adversarial robustness strategies, including adversarial training and model regularization, will be investigated to strengthen the model's resistance to evasion and poisoning attacks. Finally, optimization for edge deployment through techniques such as quantization and pruning will be pursued to achieve real-time, resource-efficient intrusion detection suitable for practical IoT applications.

CRedit authorship contribution statement

Sarra Cherfi: Conceptualization, Methodology, Software, Validation, Formal analysis, Writing – Original Draft, Software, Investigation, Visualization, Writing – Review & Editing, Project administration.; **Ammar Boulaiche:** Conceptualization, Methodology, Validation, Writing – Review & Editing, Supervision.; **Ali Lemouari:** Conceptualization, Methodology, Validation, Writing – Review & Editing, Supervision.; **Abdelhafid Abouaissa:** Conceptualization, Methodology, Validation, Writing – Review & Editing, Supervision..

Data availability

Data will be made available on request.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

- Ahmad, B., Wu, Z., Huang, Y., & Rehman, S. U. (2024). Enhancing the security in iot and IIot networks: An intrusion detection scheme leveraging deep transfer learning. *Knowledge-Based Systems*, 305, 112614.
- Ahmad, I., Wan, Z., & Ahmad, A. (2023). A big data analytics for DDOS attack detection using optimized ensemble framework in internet of things. *Internet of Things*, 23, 100825.
- Altunay, H. C., & Albayrak, Z. (2023). A hybrid CNN + LSTM-based intrusion detection system for industrial iot networks. *Engineering Science and Technology, an International Journal*, 38, 101322.
- Anitha, T., Aanankumar, S., Poonkuntran, S., & Nayyar, A. (2023). A novel methodology for malicious traffic detection in smart devices using BI-LSTM-CNN-dependent deep learning methodology. *Neural Computing and Applications*, 35(27), 20319–20338.
- Asghari, P., Rahmani, A. M., & Javadi, H. H. S. (2019). Internet of things applications: A systematic review. *Computer Networks*, 148, 241–261.
- Aversano, L., Bernardi, M. L., Cimiliti, M., & Pecori, R. (2021). A systematic review on deep learning approaches for iot security. *Computer Science Review*, 40, 100389.
- Chee, K. O., Ge, M., Bai, G., & Kim, D. D. (2024). IoTSecsim: A framework for modelling and simulation of security in internet of things. *Computers & Security*, 136, 103534.
- Díaz, M., Martín, C., & Rubio, B. (2016). State-of-the-art, challenges, and open issues in the integration of internet of things and cloud computing. *Journal of Network and Computer applications*, 67, 99–117.
- Díaz-Verdejo, J., Muñoz-Calle, J., Estepa Alonso, A., Estepa Alonso, R., & Madinabeitia, G. (2022). On the detection capabilities of signature-based intrusion detection systems in the context of web attacks. *Applied Sciences*, 12(2), 852.
- Du, J., Yang, K., Hu, Y., & Jiang, L. (2023). Nids-cnnlstm: Network intrusion detection classification model based on deep learning. *IEEE Access*, 11, 24808–24821.
- Fraihat, S., Makhadmeh, S., Awad, M., Al-Betar, M. A., & Al-Redhaei, A. (2023). Intrusion detection system for large-scale iot netflow networks using machine learning with modified arithmetic optimization algorithm. *Internet of Things*, 22, 100819.
- Ghaffari, A., Jelodari, N., pouralish, S., derakhshanfard, N., & Arasteh, B. (2024). Securing internet of things using machine and deep learning methods: A survey. *Cluster computing*, 27, 1–25.
- Issa, W., Moustafa, N., Turnbull, B., Sohrabi, N., & Tari, Z. (2023). Blockchain-based federated learning for securing internet of things: A comprehensive survey. *ACM Computing Surveys*, 55(9), 1–43.
- Javeed, D., Gao, T., Saeed, M. S., & Kumar, P. (2023). An intrusion detection system for edge-envisioned smart agriculture in extreme environment. *IEEE Internet of Things Journal*, 11(16), 26866–26876.
- Kaur, A. (2024). Intrusion detection approach for industrial internet of things traffic using deep recurrent reinforcement learning assisted federated learning. *IEEE Transactions on Artificial Intelligence*, 6(1), 37–50.
- Khan, N. W., Alshehri, M. S., Khan, M. A., Almakdi, S., Moradpoor, N., Alazeb, A., Ullah, S., Naz, N., & Ahmad, J. (2023). A hybrid deep learning-based intrusion detection system for iot networks. *Mathematical Biosciences and Engineering : MBE*, 20(8), 13491–13520.
- Kumar, G. S. C., Kumar, R. K., Kumar, K. P. V., Sai, N. R., & Brahmaiah, M. (2024). Deep residual convolutional neural network: An efficient technique for intrusion detection system. *Expert Systems with Applications*, 238, 121912.
- Lazzarini, R., Tianfield, H., & Charissis, V. (2023). A stacking ensemble of deep learning models for iot intrusion detection. *Knowledge-Based Systems*, 279, 110941.
- Liao, H., Murah, M. Z., Hasan, M. K., Aman, A. H. M., Fang, J., Hu, X., & Khan, A. U. R. (2024). A survey of deep learning technologies for intrusion detection in internet of things. *IEEE Access*, 12, 4745–4761.
- Liao, H.-J., Lin, C.-H. R., Lin, Y.-C., & Tung, K.-Y. (2013). Intrusion detection system: A comprehensive review. *Journal of Network and Computer Applications*, 36(1), 16–24.
- Lilhare, U. K., Manoharan, P., Simaiya, S., Alroobaea, R., Alsafyani, M., Baqasah, A. M., Dalal, S., Sharma, A., & Raahemifar, K. (2023). Hidm: Hybrid intrusion detection model for industry 4.0 networks using an optimized cnn-lstm with transfer learning. *Sensors*, 23(18), 7856.
- Liu, L., Xu, B., Zhang, X., & Wu, X. (2018). An intrusion detection method for internet of things based on suppressed fuzzy clustering. *EURASIP Journal on Wireless Communications and Networking*, 2018(1), 1–7.
- Lu, Y., Chai, S., Suo, Y., Yao, F., & Zhang, C. (2024). Intrusion detection for industrial internet of things based on deep learning. *Neurocomputing*, 564, 126886.
- Manocchio, L. D., Layeghy, S., Lo, W. W., Kulatililke, G. K., Sarhan, M., & Portmann, M. (2024). Flowtransformer: A transformer framework for flow-based network intrusion detection systems. *Expert Systems with Applications*, 241, 122564.
- Mesadiou, F., Torre, D., & Chennamaneni, A. (2024). Leveraging deep reinforcement learning technique for intrusion detection in SCADA infrastructure. *IEEE Access*, 12, 63381–63399.
- Mohamed, R. H., Mosa, F. A., & Sadek, R. A. (2022). Efficient intrusion detection system for iot environment. *International Journal of Advanced Computer Science and Applications*, 13(4), 572–578.
- Mohammadpour, L., Ling, T. C., Liew, C. S., & Aryanfar, A. (2022). A survey of CNN-based network intrusion detection. *Applied Sciences*, 12(16), 8162.
- Moustafa, N. (2021). A new distributed architecture for evaluating AI-based security systems at the edge: Network TON_iiot datasets. *Sustainable Cities and Society*, 72, 102994.
- Nadiah, A.-N., Alamri, A., Aljuhani, A., & Kumar, P. (2025). Transformer-based knowledge distillation for explainable intrusion detection system. *Computers & Security*, 154, 104417.
- Nandanwar, H., & Katarya, R. (2024). Deep learning enabled intrusion detection system for industrial IOT environment. *Expert Systems with Applications*, 249, 123808.
- Nguyen, D.-T., & Le, K.-H. (2023). The robust scheme for intrusion detection system in internet of things. *Internet of Things*, 24, 100999.
- Ninu, S. B. (2023). An intrusion detection system using exponential henry gas solubility optimization based deep neuro fuzzy network in MANET. *Engineering Applications of Artificial Intelligence*, 123, 105969.
- Osa, E., Orukpe, P. E., & Iruansi, U. (2024). Design and implementation of a deep neural network approach for intrusion detection systems. *e-Prime-Advances in Electrical Engineering, Electronics and Energy*, 7, 100434.
- Roy, S., Li, J., Choi, B.-J., & Bai, Y. (2022). A lightweight supervised intrusion detection mechanism for iot networks. *Future Generation Computer Systems*, 127, 276–285.
- Samantaray, M., Barik, R. C., & Biswal, A. K. (2024). A comparative assessment of machine learning algorithms in the iot-based network intrusion detection systems. *Decision Analytics Journal*, 11, 100478.
- Sharafaldin, I., Lashkari, A. H., Ghorbani, A. A. et al. (2018). Toward generating a new intrusion detection dataset and intrusion traffic characterization. *ICISSp*, 1, 108–116.
- Tang, C., Luktarhan, N., & Zhao, Y. (2020). An efficient intrusion detection method based on lightGBM and autoencoder. *Symmetry*, 12(9), 1458.
- Tareq, I., Elbagoury, B. M., El-Regaily, S., & El-Horbaty, E.-S. M. (2022). Analysis of ton-iiot, unwnb15, and edge-iiot datasets using dl in cybersecurity for iot. *Applied Sciences*, 12(19), 9572.
- Vishwakarma, M., & Kesswani, N. (2023). A new two-phase intrusion detection system with naïve bayes machine learning for data classification and elliptic envelop method for anomaly detection. *Decision Analytics Journal*, 7, 100233.
- Wang, S., Xu, W., & Liu, Y. (2023). Res-tranBiLSTM: An intelligent approach for intrusion detection in the internet of things. *Computer Networks*, 235, 109982.
- Zhang, J., Peter, J. D., Shankar, A., & Viriyasitavat, W. (2024). Public cloud networks oriented deep neural networks for effective intrusion detection in online music education. *Computers and Electrical Engineering*, 115, 109095.
- Zhang, X., Hao, L., Gui, G., Wang, Y., Adebisi, B., & Sari, H. (2023). An automatic and efficient malware traffic classification method for secure internet of things. *IEEE Internet of Things Journal*, 11(5), 8448–8458.